La Madriguera del Conejo

Diseño y análisis del motor algorítmico de un videojuego
Trabajo en parejas · Duración una semana
Dr. Román Anselmo Mora Gutiérrez

El reto de diseño

Un conejo debe encontrar su madriguera en un bosque. El equipo de desarrollo necesita construir mapas de dificultad fácil, media y difícil, y elegir cómo buscar una ruta en cada uno. Su tarea es diseñar, implementar y justificar una estrategia que combine, cuando sea pertinente, recursividad, divide y vencerás y vuelta atrás.

La pregunta central es: ¿qué combinación permite generar y resolver los tres niveles con corrección y un costo que puedan explicar y medir? No basta con nombrar paradigmas ni mostrar que el conejo llegó a la meta.

Qué deben demostrar

Al terminar, deberán explicar una descomposición real en subproblemas; escribir y justificar casos base y progreso de la recursión; identificar decisiones, restricciones y estado que se restaura al retroceder; calcular complejidad temporal y espacial; y contrastar esas expresiones con mediciones reproducibles.

Alcance del prototipo

Entreguen un programa ejecutable que muestre el mapa, S, la madriguera M y la ruta obtenida. Una representación en consola o una imagen generada por el programa es suficiente. El trabajo se concentra en el motor algorítmico: generar mapas, buscar, verificar y medir. La presentación gráfica no sustituye la evidencia algorítmica.

Cada celda es libre u obstáculo. El conejo se mueve arriba, abajo, izquierda y derecha, sin diagonales, sin salir del mapa y sin atravesar obstáculos. Cada movimiento cuesta una unidad. S y M deben estar en celdas libres. La longitud de una ruta es su número de movimientos, no el número de celdas que contiene.

Mantengan estas reglas en los tres niveles para comparar las estrategias. Pueden proponer otra mecánica como extensión después de completar el núcleo; deberán entonces ampliar el estado y revisar sus cotas.

Decisión que deberá cerrar el proyecto

Recomienden una configuración para cada nivel: cómo se construye el mapa, qué algoritmo resuelve la ruta, qué garantía ofrece y qué costo tiene. Una combinación puede usar divide y vencerás en la generación y otra técnica en la búsqueda; identifiquen ambos módulos sin atribuir al buscador la complejidad del generador.

Tres niveles y una dificultad que se comprueba

La dificultad es una propiedad del diseño del juego y de su interacción con el algoritmo. Un mapa más grande o con más obstáculos no es automáticamente más difícil. Primero comparen mapas del mismo tamaño; después estudien qué cambia al aumentar n.

Nivel	Intención de diseño	Evidencia que deben reunir
Fácil	Ruta corta, pocas decisiones relevantes y pocos desvíos	Mapa, ruta verificada y cantidad de estados explorados
Medio	Decisiones que conducen a callejones sin salida y una ruta menos directa	Traza de al menos un fracaso, regreso y elección de otra rama
Difícil	Más alternativas y ciclos; ramas que parecen útiles pero obligan a reconsiderar decisiones	Más trabajo para al menos una estrategia bajo las mismas reglas de medición; expliquen si otra estrategia lo resuelve con facilidad

Diseñen primero tres mapas 9 × 9, uno por nivel, con S y M identificadas. Mantengan un orden de exploración común: arriba, derecha, abajo, izquierda. No cambien el orden para favorecer un resultado; si estudian otro orden, traten ese cambio como un experimento separado.

Documenten para cada mapa: tamaño, número de celdas libres, densidad de obstáculos, longitud mínima según BFS y estados expandidos por cada método. Incluyan el número de callejones sin salida o un indicador de ramificación que definan de manera verificable. Son indicadores del diseño, no una fórmula universal de dificultad.

La decisión de diseño también se evalúa

Si el nivel difícil no requiere más trabajo que el medio para el algoritmo que eligieron, expliquen la causa y ajusten el diseño o reconozcan el resultado. No cambien las etiquetas después de observar los tiempos sin registrar la modificación.

Entrada y salida del motor

La entrada común es el mapa, S, M y los límites de ejecución. La salida debe distinguir: ruta encontrada, inexistencia de ruta demostrada y búsqueda interrumpida por límite. Si se obtiene una ruta, devuelvan la lista ordenada de posiciones, su longitud y las métricas de búsqueda.

Una ruta hallada antes de agotar el límite puede ser factible sin que se haya probado que es mínima. Reporten por separado “existe una ruta” y “se certificó su optimalidad”. Un algoritmo interrumpido no puede declarar que el mapa carece de solución.

Los algoritmos que deben construir y contrastar

1 Dos referencias para entender el problema

Implementen BFS con cola y DFS recursivo con marcas globales de visitado. Ambos buscan una ruta; BFS también proporciona la longitud mínima cuando todos los movimientos cuestan uno. DFS no garantiza una ruta mínima. Reconstruyan las rutas con predecesores o con una estructura equivalente.

En DFS de alcanzabilidad, una celda ya explorada conserva su marca. Volver al llamado anterior no exige borrar esa información. Documenten los casos base, la forma de evitar ciclos y el evento que garantiza el progreso hacia la terminación.

2 Vuelta atrás para optimizar una ruta

Construyan una búsqueda de caminos simples que intente encontrar la ruta más corta. Mantengan la ruta parcial, las celdas de esa ruta y la mejor solución conocida. Al elegir un vecino, agréguenlo; al regresar, retiren el vecino y restauren el estado de la ruta. Una celda puede reaparecer en otra rama, pero no repetirse dentro de la ruta actual.

Al alcanzar M, actualicen la mejor solución si mejora. Si la longitud parcial ya es mayor o igual que la mejor longitud conocida, poden esa rama. Pueden añadir la distancia Manhattan como cota inferior, pero deben justificar por qué no sobreestima los pasos restantes. No utilicen el resultado óptimo de BFS como respuesta precargada de esta búsqueda.

Comparen este método con DFS: ambos pueden explorar en profundidad, pero tienen distintos objetivos, marcas y cantidades de estados examinados. Entreguen una traza que muestre elegir, explorar, detectar fracaso o poda, deshacer y probar otra opción.

3 Divide y vencerás con una función concreta

Implementen un generador recursivo de mapas. Una opción es dividir una región en dos subregiones, construir sus interiores y unirlos mediante un paso válido. Otra es construir cuatro regiones y conectar sus fronteras. Definan cómo se eligen cortes y pasos y cómo evitan bloquear conexiones ya establecidas.

Indiquen qué devuelve cada subproblema, cuál es el caso base, cómo disminuye el tamaño y por qué la combinación conserva la conexión entre las regiones. Distingan el costo de inicializar la matriz, construir paredes, abrir pasos y validar la conectividad. Para crear alternativas pueden retirar paredes de manera controlada después de la construcción; midan ese trabajo por separado.

Pueden aplicar divide y vencerás al solucionador en lugar del generador si justifican las entradas y salidas de cada región, todas las conexiones necesarias y la corrección de la combinación. Dibujar cuadrantes o explorar vecinos recursivamente no basta para acreditar este paradigma. Para una semana se recomienda la ruta del generador.

Corrección y cálculo de complejidad

Un análisis por cada módulo

Presenten pseudocódigo de BFS, DFS recursivo, vuelta atrás y generación por divide y vencerás. Definan parámetros, estructuras, resultado y operación contada. El pseudocódigo debe describir su implementación, incluidas marcas, restauraciones, podas y condiciones de salida.

Para cada módulo argumenten: por qué una salida es válida, por qué termina y qué garantiza. En la vuelta atrás, expliquen por qué las podas no eliminan una posible mejora. En el generador, prueben por inducción o mediante un argumento equivalente que las conexiones propuestas permiten alcanzar las regiones construidas.

El tamaño no es siempre la misma cantidad

$$\mathrm{n} = \text{lado de la rejilla}; \qquad \mathrm{A} = \mathrm{n}^2; \qquad \mathrm{V} = \text{celdas libres}; \qquad \mathrm{E} = \text{adyacencias libres}$$

Usen un grafo no dirigido con una arista por par de celdas libres vecinas. Entonces $V \le n^2$ y $E \le 2n(n-1)$. En particular $E = O(V)$, porque cada celda tiene grado a lo sumo cuatro. No cuenten cada arista dos veces y luego la presenten como dos aristas diferentes.

Con listas de vecinos implícitas y estructuras adecuadas, BFS y DFS tienen costo $O(V+E)$ en el peor caso de búsqueda y memoria $O(V)$. Si inicializan o copian matrices completas, incluyan $O(n^2)$ en el costo total. Una pila recursiva de DFS puede alcanzar $O(V)$ de profundidad; una llamada por celda no significa una profundidad de n.

Recurrencias derivadas de lo que programaron

Escriban una recurrencia del generador e identifiquen cantidad y tamaño de subproblemas y costo de división y combinación. Resuélvanla mediante árbol, sustitución o Teorema Maestro cuando se cumplan sus condiciones. No asignen automáticamente $T(n) = 4T(n/2) + \Theta(n)$ por dibujar cuatro cuadrantes.

Por ejemplo, esa recurrencia sí produciría $\Theta(n^2)$, pero sólo describe un diseño con cuatro subproblemas de lado $n/2$ y un trabajo adicional $\Theta(n)$. Copiar submatrices completas o probar muchas conexiones cambia la recurrencia. Si trabajan con el área A, no la confundan con el lado n.

El costo de explorar alternativas

En la vuelta atrás la profundidad de una ruta simple no excede V y hay hasta cuatro vecinos por decisión. Una cota superior conservadora $O(V \cdot 4^V)$ admite explorar muchas rutas y copiar soluciones de longitud $O(V)$; no pretende ser una cota ajustada. Obtengan y expliquen una cota coherente con su código y sus podas. Usar marcas de la ruta evita ciclos, pero no evita explorar exponencialmente muchas rutas distintas.

Sin ninguna prevención de ciclos, una búsqueda puede no terminar. No describan ese caso simplemente como “exponencial”. La notación O expresa una cota; no implica que al duplicar n el tiempo medido deba multiplicarse exactamente por cuatro.

Pruebas y mediciones reproducibles

Primero comprueben que funciona

Construyan pruebas pequeñas: S coincide con M; una ruta única; varias rutas; un callejón sin salida; un ciclo; y una meta inaccesible. Para cada ruta verifiquen inicio, final, pasos adyacentes, ausencia de obstáculos y longitud. En estos mapas contrasten la longitud óptima de vuelta atrás con BFS cuando la búsqueda termina sin interrupción.

Después comparen sobre los mismos mapas

Generen 60 mapas: tamaños 9, 17, 33 y 65; tres niveles; cinco semillas por combinación de tamaño y nivel. Conserven mapas y semillas, además de las reglas y parámetros de generación. Todos los buscadores deben recibir el mismo mapa y una copia independiente de su estado de trabajo.

Ejecuten BFS y DFS sobre los 60 mapas. Ejecuten vuelta atrás al menos sobre los 15 mapas de tamaño 9 × 9. Establezcan para esta búsqueda un máximo de 100 000 llamadas o 2 segundos por ejecución, lo que ocurra primero. No es obligatorio ejecutarla en mapas mayores; si lo hacen, conserven los mismos límites y reporten interrupciones.

Midan el generador por separado sobre las 60 instancias y verifiquen la conectividad con BFS fuera de su tiempo de generación. No atribuyan el tiempo de validación al algoritmo de generación sin identificarlo. Si la generación exige rechazar candidatos, registren los intentos y su costo.

Qué significa cada medición

Métrica	Definición
Estados expandidos	Veces que se procesa un estado para examinar sus vecinos; en vuelta atrás una celda puede contarse muchas veces
Celdas distintas	Número de posiciones diferentes alcanzadas; no es igual a expansiones ni al total de operaciones
Operaciones y retrocesos	Cuenten por separado vecinos examinados, llamadas, podas y restauraciones efectivas de la ruta
Memoria de búsqueda	Máximo de elementos en cola o pila y tamaño de marcas; indiquen si miden elementos o bytes
Ruta y estado final	Longitud, factibilidad, optimalidad certificada y finalización o límite
Tiempo	Tiempo del buscador, sin dibujo, lectura ni generación; indiquen si incluye inicializar sus estructuras

Utilicen un reloj de alta resolución y tres repeticiones por ejecución completa; reporten la mediana del tiempo. El conteo de operaciones debe reiniciarse y coincidir si el algoritmo es determinista. Para búsquedas que alcanzan un límite basta un intento: registren el límite, el trabajo alcanzado y la mejor ruta factible disponible. No mezclen estos tiempos con los de soluciones terminadas.

Análisis y estrategia final del videojuego

De los registros a las conclusiones

Entreguen un archivo de resultados con una fila por ejecución: identificador de mapa, tamaño, nivel, semilla, celdas libres, obstáculos, algoritmo, repetición, tiempo, expansiones, celdas distintas, vecinos examinados, profundidad o frontera máxima, llamadas, retrocesos, podas, longitud y estado final. Utilicen “no aplica” cuando la métrica no corresponda.

Grafiquen expansiones y tiempo contra tamaño, separados por nivel y algoritmo. Con sólo cuatro tamaños, una curva visual no demuestra una clase asintótica: contrasten con el análisis del pseudocódigo. Expliquen por qué conteos similares pueden producir tiempos distintos y por qué más obstáculos pueden reducir la región alcanzable.

Comparen el cociente entre longitud de la ruta y longitud óptima de BFS cuando ambas existan y esta última sea positiva. Si S = M, reporten longitud cero sin dividir. No asignen una longitud ficticia a mapas sin solución o búsquedas interrumpidas sin una ruta disponible.

Elijan una arquitectura y defiéndanla

Presenten una tabla con tres filas, una por nivel: construcción del mapa, buscador elegido, garantía de la ruta, tiempo y memoria observados, y evidencia que apoya la elección. Pueden usar el mismo buscador en los tres niveles si los resultados lo justifican; no inventen una ventaja para obligar a usar un algoritmo diferente en cada uno.

Expliquen dónde intervienen recursividad, divide y vencerás y vuelta atrás dentro del proyecto. Las técnicas pueden combinarse entre módulos y no tienen que aparecer todas en cada nivel. BFS puede ser la elección final aunque no sea recursivo; su papel como referencia no sustituye las otras implementaciones solicitadas.

Preguntas que deben responder con sus resultados

¿Qué hace distinto al nivel difícil respecto del medio cuando ambos tienen igual tamaño? ¿Cuál algoritmo encuentra una ruta y cuál garantiza la más corta? ¿Qué estado se restaura en vuelta atrás? ¿Qué trabajo ahorra una poda y qué garantía conserva? ¿Qué se combina en divide y vencerás? ¿Qué recurrencia describe esa combinación? ¿Qué costo domina al aumentar el mapa y qué límites aparecen en la pila recursiva?

Qué no pueden concluir de las mediciones

Una muestra de mapas no demuestra el costo promedio de todos los bosques posibles. El peor tiempo observado no equivale a una prueba de peor caso asintótico. Si dos métodos comparten $O(n^2)$, eso no obliga a que tengan el mismo tiempo o la misma calidad de ruta. Y clasificar un mapa como difícil no cambia por sí solo la clase de complejidad del algoritmo.

Una semana en parejas y entrega

Día	Avance verificable
1	Modelen el problema, acuerden qué significa dificultad y dibujen los tres mapas 9 × 9
2	Implementen y verifiquen BFS y DFS; registren casos base, marcas y reconstrucción
3	Implementen el generador por divide y vencerás y deriven su recurrencia
4	Implementen vuelta atrás con restauración y poda; prueben casos pequeños y límites
5	Ejecuten la batería sobre los mismos mapas y conserven archivos reproducibles
6	Analicen las gráficas, comparen con las cotas y elijan una estrategia por nivel
7	Revisen resultados, ejecuten desde cero y entreguen informe, código y demostración

Ambos integrantes deben programar y revisar. Intercambien responsabilidades a mitad de la semana. Cada persona explicará al menos una recurrencia, una decisión de restauración y una comparación experimental. Incluyan contribuciones individuales verificables; no basta con separar “programación” y “redacción”.

Entregables

Un informe técnico de seis a ocho páginas, más anexos: modelo y reglas; diseño de los tres niveles; algoritmos y corrección; cálculo de complejidad y recurrencia; experimento y resultados; y arquitectura recomendada. Toda tabla y figura debe tener título, unidades e interpretación.

Anexen código y un archivo de instrucciones para ejecutarlo, dependencias, mapas, semillas y registros de resultados. Incluyan tres capturas o salidas representativas y una traza de vuelta atrás. Entreguen una demostración grabada de hasta tres minutos del prototipo y de la selección de niveles. No se exige desarrollar gráficos, sonido o un videojuego comercial completo.

Criterio	Puntos
Diseño del juego y tres niveles justificados	1.5
Implementación y corrección de las estrategias	2.5
Cálculo de complejidad temporal y espacial y solución de recurrencia	2.5
Experimento reproducible y comparación entre teoría y resultados	2.0
Comunicación, contribuciones individuales y ejecución documentada	1.5
Total	10.0

En cada criterio, la puntuación completa requiere evidencia correcta y reproducible; se otorga crédito parcial por componentes demostrados. Nombrar una técnica sin implementarla y justificarla no acredita ese componente. Una búsqueda interrumpida, correctamente reportada y analizada, es un resultado válido del experimento.

Pueden utilizar IA para consultar, depurar o revisar, pero deben declarar herramienta, uso y verificación. Cada integrante debe poder explicar y modificar el código entregado. Prioricen argumentos y mediciones sobre adornos visuales.